

SCSS Maps (Ассоциативные Массивы)

SCSS Maps (или ассоциативные массивы) позволяют хранить наборы именованных значений. Они полезны для организации связанных стилей и данных, таких как цветовые палитры, шрифты или размеры.

Определение Map

Map определяется с использованием круглых скобок `()` и содержит пары ключ-значение, разделенные двоеточием `:`. Пары разделяются запятыми `,`.

Синтаксис:

```
$map-name: (  
  key1: value1,  
  key2: value2,  
  key3: value3,  
  // ...  
);
```

Пример:

```
$colors: (  
  primary: #007bff,  
  secondary: #6c757d,  
  success: #28a745,  
  danger: #dc3545,  
  warning: #ffc107,  
  info: #17a2b8  
);  
  
$font-sizes: (  
  small: 12px,  
  medium: 16px,  
  large: 20px,  
  xlarge: 24px  
);
```

Доступ к Значениям Map (`map-get()`)

Для получения значения из Map используется встроенная функция `map-get(<map>, <key>)`.

Синтаксис:

```
map-get($map-name, key);
```

Пример:

```
$primary-color: map-get($colors, primary); // $primary-color будет #007bff
$large-font:    map-get($font-sizes, large); // $large-font будет 20px

.button-primary {
  background-color: $primary-color;
  color: white;
  font-size: map-get($font-sizes, medium);
}

.heading-large {
  font-size: $large-font;
}
```

Проверка наличия Ключа (map-has-key())

Функция `map-has-key(<map>, <key>)` возвращает `true`, если указанный ключ существует в Map, и `false` в противном случае.

Синтаксис:

```
map-has-key($map-name, key);
```

Пример:

```
@if map-has-key($colors, success) {
  .alert-success {
    background-color: map-get($colors, success);
    color: white;
  }
} @else {
  .alert-success {
    background-color: lightgreen; // Fallback
    color: darkgreen;
  }
}
```

Получение Списка Ключей (map-keys())

Функция `map-keys(<map>)` возвращает список всех ключей, содержащихся в Map.

Синтаксис:

```
map-keys($map-name); // Возвращает список: key1, key2, key3, ...
```

Пример:

```
$theme-colors: map-keys($colors); // $theme-colors будет: primary, secondary,
success, danger, warning, info

@each $color-name in $theme-colors {
  .btn-#{ $color-name } {
    background-color: map-get($colors, $color-name);
    color: white;
  }
}
```

Получение Списка Значений (map-values())

Функция `map-values(<map>)` возвращает список всех значений, содержащихся в Map.

Синтаксис:

```
map-values($map-name); // Возвращает список: value1, value2, value3, ...
```

Пример:

```
$all-font-sizes: map-values($font-sizes); // $all-font-sizes будет: 12px, 16px,
20px, 24px

body {
  font-size: nth($all-font-sizes, 2); // Используем средний размер шрифта по
умолчанию (16px)
}
```

Получение Количества Ключ-Значение Пар (map-length())

Функция `map-length(<map>)` возвращает количество пар ключ-значение в Map.

Синтаксис:

```
map-length($map-name); // Возвращает число
```

Пример:

```
$num-colors: map-length($colors); // $num-colors будет 6

.color-palette-info::after {
  content: "Содержит " + $num-colors + " цвета.";
}
```

Объединение Maps (`map-merge()`)

Функция `map-merge(<map1>, <map2>)` объединяет две Map в новую Map. Если в обеих Map есть одинаковые ключи, значение из второй Map перезапишет значение из первой.

Синтаксис:

```
map-merge($map-name1, $map-name2); // Возвращает новую объединенную Map
```

Пример:

```
$base-fonts: (
  primary: 'Arial',
  secondary: 'Helvetica'
);

$extended-fonts: (
  tertiary: 'Verdana',
  primary: 'Arial Neue' // Перезапишет значение 'primary'
);

$all-fonts: map-merge($base-fonts, $extended-fonts);
// $all-fonts будет: (primary: 'Arial Neue', secondary: 'Helvetica', tertiary: 'Verdana')
```

Удаление Ключей из Map (`map-remove()`)

Функция `map-remove(<map>, <key1>, <key2>, ...)` возвращает новую Map, из которой удалены указанные ключи.

Синтаксис:

```
map-remove($map-name, key1, key2, ...); // Возвращает новую Map без указанных ключей
```

Пример:

```
$social-icons: (  
  facebook: 'fa-facebook',  
  twitter: 'fa-twitter',  
  instagram: 'fa-instagram',  
  pinterest: 'fa-pinterest'  
);  
  
$essential-icons: map-remove($social-icons, pinterest);  
// $essential-icons будет: (facebook: 'fa-facebook', twitter: 'fa-twitter',  
instagram: 'fa-instagram')
```

Использование Maps в Циклах (@each)

Maps часто используются вместе с директивой `@each` для итерации по их ключам и значениям.

Синтаксис:

```
@each $key, $value in $map-name {  
  // Стили, применяемые для каждой пары ключ-значение  
  .#{$key} {  
    property: $value;  
  }  
}
```

Пример:

```
$spacings: (  
  small: 5px,  
  medium: 10px,  
  large: 15px,  
  xlarge: 20px  
);  
  
@each $size, $value in $spacings {  
  .margin-#{$size} {  
    margin: $value;  
  }  
  
  .padding-#{$size} {  
    padding: $value;  
  }  
}
```

Этот пример создаст классы `.margin-small`, `.margin-medium`, `.padding-small`, `.padding-medium` и так далее, с соответствующими значениями от Map `$spacings`.

Заключение

SCSS Maps предоставляют удобный способ организации и управления наборами связанных значений. Использование функций `map-get()`, `map-has-key()`, `@each` и других позволяет создавать более гибкий, масштабируемый и поддерживаемый CSS код. Они особенно полезны при работе с темами, цветовыми палитрами и наборами типографики.